

Quality-Aware Incremental Path Planning using D* Lite

1. Overview

This work develops an incremental path planner based on the D* Lite algorithm for navigation in a dynamically changing environment. Unlike conventional shortest-path planning, where path length or traversal cost is the sole consideration, the proposed approach incorporates two additional properties of a path: *transition reliability* and *distance from obstacles*. The objective is to provide a planner capable of efficiently replanning when the environment changes while allowing a controllable trade-off between path cost and path quality.

The underlying D* Lite algorithm is retained as the incremental search mechanism. In particular, the planner maintains the persistent g -values, rhs -values, priority queue, and k_m term across environment updates. Consequently, when an edge is added, removed, or modified, the planner updates only the affected vertices and propagates the resulting inconsistency instead of rebuilding the entire search tree.

2. Problem Formulation

Let the environment be represented by a directed graph

$$G = (V, E),$$

where each vertex $u \in V$ represents a state and each directed edge $(u, v) \in E$ represents a feasible transition. Each transition has a non-negative traversal cost

$$c(u, v) \geq 0$$

and a reliability

$$r(u, v) \in [0, 1].$$

Let s and g denote the current start and goal states, respectively. A path is represented by

$$P = (s_0, s_1, \dots, s_n),$$

where $s_0 = s$ and $s_n = g$.

The conventional path cost is

$$C(P) = \sum_{i=0}^{n-1} c(s_i, s_{i+1}).$$

The cumulative reliability of a path is modelled as the probability that all transitions along the path succeed:

$$R(P) = \prod_{i=0}^{n-1} r(s_i, s_{i+1}).$$

This formulation naturally penalizes paths containing unreliable transitions.

3. Obstacle Clearance and Safety

Let $B \subseteq V$ denote the set of obstacle or otherwise undesirable states. For every state u , define its distance from the nearest obstacle as

$$D(u) = \min_{b \in B} d(u, b),$$

where $d(\cdot, \cdot)$ denotes the distance in the Cartesian embedding of the states.

To make the distance dimensionless and comparable with reliability, it is normalized as

$$S(u) = \frac{D(u)}{D_{\max}},$$

where $D_{\max}$ is the maximum relevant clearance in the environment. Thus,

$$0 \leq S(u) \leq 1.$$

A state located directly on an obstacle has $D(u) = 0$ and consequently $S(u) = 0$. Such states are treated as invalid states for traversal rather than merely receiving a low preference.

For a complete path, safety is conservatively represented by its minimum state safety:

$$S(P) = \min_{u \in P} S(u).$$

This choice reflects the fact that a path containing a single highly unsafe section should not be considered highly safe merely because the remainder of the path has large clearance.

4. Quality Model

Reliability and obstacle clearance are combined into a normalized path-quality score:

$$Q(P) = \alpha R(P) + \beta S(P)$$

where

$$\alpha, \beta \geq 0, \quad \alpha + \beta = 1.$$

The parameters α and β control the relative preference for reliability and obstacle clearance. Increasing α favors paths with higher transition reliability, while increasing β favors paths that maintain greater distance from obstacles.

The quality score is therefore not an independent replacement for the D* Lite cost function. Instead, it provides a mechanism for expressing the desired trade-off between conventional path cost and additional path characteristics.

5. D* Lite Formulation

The core incremental search mechanism follows standard D* Lite. For every state u , let $g(u)$ denote the current cost-to-go estimate and let $rhs(u)$ denote the one-step lookahead value. For $u \neq g$,

$$rhs(u) = \min_{v \in Succ(u)} [c(u, v) + g(v)]$$

while

$$rhs(g) = 0.$$

A vertex is consistent when

$$g(u) = rhs(u).$$

Changes to transition costs or connectivity can make vertices inconsistent. These vertices are inserted into the priority queue and the inconsistency is propagated through predecessor relationships using **UpdateVertex**. The shortest-path computation terminates when the start state is locally consistent and no queue entry has a key smaller than the key of the start state.

The standard D* Lite key is

$$k_1(u) = \min(g(u), rhs(u)) + h(s, u) + k_m,$$

where h is the heuristic and k_m accounts for changes in the start state.

6. Quality-Aware Priority

To incorporate the proposed quality information without modifying the Bellman backup, quality is introduced at the priority level rather than replacing the additive cost model. A quality-aware secondary priority component can be defined as

$$k_2(u) = \frac{\min(g(u), rhs(u))}{\max(Q_u, \epsilon)},$$

where Q_u is the quality estimate associated with the currently considered continuation through u and $\epsilon > 0$ is a small constant preventing division by zero.

The resulting key is compared lexicographically:

$$\boxed{Key(u) = (k_1(u), k_2(u))}.$$

The first component preserves the conventional D* Lite ordering based on cost-to-go and heuristic information, while the second component provides a preference toward higher-quality alternatives when comparing otherwise promising states.

Importantly, this modification does **not** redefine the D* Lite Bellman equation as a ratio such as $(c + g)/Q$. The additive cost structure is retained in rhs , preserving the mathematical structure required by the incremental shortest-path algorithm. The quality-aware component is therefore treated as a search-priority preference rather than as a claim of exact optimality with respect to a non-additive objective.

7. Dynamic Replanning

The principal advantage of the proposed system is its ability to operate in a changing environment. The planner is initialized only once. During subsequent planning cycles, changes such as edge insertion, edge removal, cost modification, reliability modification, or obstacle-state changes are applied to the existing graph.

Only vertices affected by these changes are updated using `UpdateVertex`. The existing g , rhs , OPEN queue, and k_m values are retained, allowing D* Lite to reuse previously computed information.

Thus, the overall process is

$$\text{Initial Environment} \rightarrow \text{D* Lite Search} \rightarrow \text{Environment Update} \rightarrow \text{Local Repair} \rightarrow \text{New Path}.$$

This avoids recomputing the entire search from scratch after every environmental change.